

Laboratorio 1

Introducción a lenguaje Ensamblador

Tiago Reino

email:tiago.reino@estudiantes.utec.edu.uy

Sofia Lapitz

email:sofia.lapitz@estudiantes.utec.edu.uy

David Santana

email:david.santana@estudiantes.utec.edu.uy

Link para acceder a los videos en Google Drive:

<https://drive.google.com/drive/folders/1S9oluBCLg4OkxOA5pSfvMtGWvjA01kLX?usp=sharing>

Link para acceder al repositorio GitHub:

<https://github.com/tiagoreino/Laboratorio-1.git>

Resumen—En el presente informe se documenta el desarrollo de cuatro sistemas embebidos implementados en lenguaje ensamblador sobre los microcontroladores ATmega328p y ATmega2560, en el marco del primer laboratorio de la unidad curricular Tecnologías de Microprocesamiento. Los sistemas desarrollados comprenden una lavadora automática controlada mediante una máquina de estados finita, un contador digital hexadecimal con display de 7 segmentos, un sistema de selección y ejecución de ocho secuencias luminosas, y una unidad aritmético-lógica (ALU) de 4 bits con banderas de estado. Para cada uno de los problemas se detallan las decisiones de diseño adoptadas respecto a la asignación de puertos de entrada/salida, el manejo del registro de estado SREG, y las estrategias de temporización y antirrebote implementadas. Todos los sistemas fueron validados mediante simulación en PicSimLab previo a su implementación física, lo que permitió identificar y corregir discrepancias entre el comportamiento simulado y el comportamiento real del hardware.

Index Terms—Microcontrolador Lenguaje ensamblador Pic-simLab ATmega328p ATmega2560 Máquinas de estados Registros

I. INTRODUCCIÓN

La programación en lenguaje ensamblador constituye una herramienta fundamental para comprender el funcionamiento interno de un microcontrolador, dado que expone de forma directa el manejo de registros, la organización de la memoria y la interacción con los periféricos de entrada/salida, sin las abstracciones propias de los lenguajes de alto nivel. Esta comprensión resulta particularmente relevante en el campo de la mecatrónica, donde el desarrollo de sistemas embebidos eficientes y confiables depende en gran medida de un manejo preciso de los recursos de hardware disponibles.

En este contexto, el presente laboratorio propone el desarrollo de cuatro sistemas de complejidad creciente sobre los microcontroladores ATmega328p y ATmega2560, abarcando desde el control de una máquina de estados con múltiples condiciones de seguridad, hasta el diseño de una unidad aritmético-lógica con manejo explícito de banderas de estado. A través de estos problemas se busca consolidar el uso práctico

de los principales recursos de la arquitectura AVR: los registros de propósito general, el registro de estado SREG, los puertos de entrada/salida, las operaciones aritméticas y lógicas, las instrucciones de salto condicional, y la implementación de retardos tanto por software como mediante temporizadores por hardware.

El presente informe se organiza de la siguiente manera: la Sección II detalla los objetivos generales y específicos perseguidos; la Sección V describe el diseño y la implementación adoptados para cada uno de los cuatro problemas propuestos; la Sección VI presenta lo obtenido junto con las principales dificultades encontradas durante el desarrollo; y finalmente, la Sección VII sintetiza los aprendizajes derivados de la experiencia.

II. OBJETIVOS

II-A. Objetivo General

En esta primera instancia de laboratorio se busca que los estudiantes se familiaricen con los aspectos fundamentales del microcontrolador ATmega328P y la programación en lenguaje ensamblador. Se trabajará con los principales recursos de la arquitectura AVR, haciendo énfasis en el uso de los registros de propósito general, el registro de estado SREG, los puertos de entrada/salida, las operaciones aritméticas y lógicas, las instrucciones de salto y la implementación de retardos mediante software.

II-B. Objetivos Específicos

- **Programar** microcontrolador en lenguaje ensamblador para cumplir tareas solicitadas.
- **Diseñar** sistemas con interfaz para operar microcontrolador.
- **Simular** sistemas antes de su aplicación física para corroborar lógica y cuidar la integridad de los componentes.

III. MATERIALES UTILIZADOS

Se utilizaron los siguientes materiales:

- Microcontrolador ATmega2560.
- Microcontrolador ATmega328P.
- Protoboard.
- Fuente de alimentación.
- LEDs.
- Resistencias.
- Pulsadores.
- Cables.

IV. MARCO TEÓRICO

El ATmega328P es un microcontrolador de 8 bits de la familia AVR, también basado en arquitectura RISC, ampliamente utilizado en placas Arduino como el Arduino Uno. Cuenta con 32 KB de memoria flash, 2 KB de SRAM, 1 KB de EEPROM, 23 líneas de entrada/salida de propósito general y 32 registros de trabajo de propósito general, lo que lo convierte en una alternativa de menor escala frente al ATmega2560, utilizada como referencia habitual en el ámbito educativo y de prototipado [1].

El ATmega2560 es un microcontrolador de 8 bits basado en la arquitectura AVR RISC (High performance Low power), lo que permite la ejecución de la mayoría de las instrucciones en un solo ciclo de reloj. Lo que le permite tener un balance entre el consumo de energía y la velocidad de procesamiento. [2].

Microchip Studio es el entorno de desarrollo integrado (IDE) utilizado para programar y depurar aplicaciones en microcontroladores AVR y SAM. Permite escribir código tanto en C/C++ como en ensamblador, y ofrece herramientas de depuración avanzadas, incluyendo la inspección de registros y memoria en tiempo real, resultando la sucesora directa del anteriormente denominado Atmel Studio [3].

PICSimLab es un emulador de placas de desarrollo en tiempo real que permite simular el comportamiento de microcontroladores AVR, entre otras arquitecturas, integrando depuración mediante *avr-gdb*. A diferencia de un entorno de edición de código, PICSimLab se centra en emular fielmente el hardware físico (incluyendo periféricos como LEDs, pulsadores y displays), permitiendo validar el funcionamiento de un programa antes de su implementación sobre un microcontrolador real [4].

Los temporizadores (*timers*) de un microcontrolador AVR son periféricos de hardware capaces de contar ciclos de reloj de forma independiente a la ejecución del programa principal. Entre sus modos de operación se encuentra el modo CTC (*Clear Timer on Compare Match*), en el cual el contador se reinicia automáticamente al alcanzar un valor de comparación definido por el usuario, generando una interrupción periódica con una frecuencia precisa y configurable mediante el preescalador y el registro de comparación correspondiente [5].

La arquitectura AVR dispone de 32 registros de propósito general de 8 bits, utilizados como espacio de trabajo principal para las operaciones del procesador, y de un registro de estado (SREG) de 8 bits que almacena información sobre el resultado de la última instrucción aritmética o lógica ejecutada, mediante

banderas individuales como Carry (C), Zero (Z) y Negativo (N), entre otras. Estas banderas son consultadas habitualmente por las instrucciones de salto condicional para determinar el flujo de ejecución del programa [6].

El rebote de contacto (*bouncing*) es un fenómeno físico propio de los interruptores y pulsadores mecánicos, en el cual los contactos metálicos oscilan brevemente entre los estados abierto y cerrado antes de estabilizarse, como consecuencia de la energía mecánica del accionamiento. Si no se corrige, este fenómeno puede ser interpretado por un sistema digital como múltiples pulsaciones en lugar de una única, por lo que resulta necesario aplicar técnicas de antirrebote, ya sea por hardware (filtros RC, biestables) o por software (retardos de estabilización), para garantizar una lectura confiable de la entrada [7].

V. METODOLOGÍA

V-A. Problema A. Lavadora Automática

Se está desarrollando una lavadora en fase final de fabricación. Antes de su lanzamiento al mercado, es necesario programar un software que controle su operación, gestione diferentes modos de funcionamiento, y permita notificar en que modo de funcionamiento se encuentra mediante LEDs.

Para el desarrollo de la solución se implementó una máquina de estados finita compuesta por cinco estados principales: *Listo*, *Lavado*, *Centrifugado*, *Secado* y *Fin*, precedidos por un sub-flujo de selección de carga. El estado *Listo* se mantiene mientras el sistema espera que el usuario seleccione el tipo de carga y confirme el inicio del ciclo; una vez confirmado el inicio, el sistema transiciona secuencialmente a través de *Lavado* → *Centrifugado* → *Secado* → *Fin*, sin posibilidad de retroceso, reflejando el comportamiento esperado de un ciclo de lavado real.

Las entradas del sistema son cuatro pulsadores/sensores conectados al puerto D, configurados como entradas con resistencias de pull-up internas habilitadas:

- **PD2:** pulsador de selección de carga, permite ciclar entre carga ligera, media y pesada.
- **PD3:** pulsador de inicio, dispara la transición del estado *Listo* a *Lavado* siempre que las condiciones de seguridad se cumplan.
- **PD4:** sensor (interruptor) de puerta cerrada, condición de seguridad obligatoria para permitir el inicio.
- **PD5:** sensor de nivel de agua, segunda condición de seguridad obligatoria.

(Para la implementación de ambos sensores se utilizaron resistencias debido a la disponibilidad de espacio y material)

El botón de inicio (PD3) solo es evaluado cuando ambas condiciones de seguridad (puerta cerrada y nivel de agua correcto) se cumplen simultáneamente, evitando que el ciclo de lavado pueda iniciarse en condiciones inseguras.

Las salidas del sistema se distribuyen entre los puertos B y C. El puerto C (PC0-PC4) concentra los cinco LEDs indicadores de estado, uno por cada estado de la máquina, activados de forma excluyente (un único LED de estado encendido a la vez). El puerto B se utiliza para los tres LEDs indicadores de la carga seleccionada (PB0-PB2: ligera, media

y pesada respectivamente) y las dos señales de control del motor (PB3 para giro hacia la derecha, PB4 para giro hacia la izquierda), utilizadas durante el estado *Secado* para alternar el sentido de giro.

Para la temporización de los distintos tiempos de espera (giro, pausa, centrifugado y secado) se utilizó el Timer1 del microcontrolador configurado en modo CTC (*Clear Timer on Compare Match*), con un preescalador de 1024 y un valor de comparación OCR1A = 15624, lo cual genera una interrupción exactamente cada 1 segundo a partir de una frecuencia de reloj de 16 MHz.

Descripción de los estados fundamentales La máquina de estados representa el funcionamiento secuencial de la lavadora, desde la selección del tipo de carga hasta la finalización del proceso. Cada estado representa una etapa específica del funcionamiento y las transiciones dependen de las condiciones de entrada, de la finalización de los tiempos establecidos y de la cantidad de ciclos realizados:

- **Selección de carga (INICIO, LIGERA, MEDIA, PESADA):** Establece la selección inicial. Mediante el selector se conmuta entre los tres tipos de carga. Al recibir la orden de inicio con la puerta cerrada (y el nivel de agua adecuado, cambia a la etapa de lavado.
- **Lavado (LAVADO INICIO GIRO MOTOR y ESPERA):** Activa el motor según el tiempo configurado (2s ligera, 3s media y 4s pesada), seguido de su respectiva pausa (1s, 2s y 3s). Repite el ciclo hasta cumplir la cantidad establecida.
- **Centrifugado (CENTRIFUGADO INICIO GIRO MOTOR y ESPERA):** Ejecuta el centrifugado continuo durante 15s (ligera), 18s (media) o 21s (pesada), seguido de una pausa para estabilizar la máquina antes de pasar al secado.
- **Secado (SECADO INICIO, GIRO DERECHA, GIRO IZQUIERDA y ESPERAS):** Secuencia de giros alternados a la derecha y a la izquierda con pausas intermedias. Ejecuta 1, 2 o 3 ciclos completos según la carga seleccionada.
- **FIN:** Estado final que indica la conclusión de la secuencia completa y mantiene la máquina detenida.

Flujo de operación y condiciones de transición El funcionamiento comienza en el estado *INICIO*. Una vez seleccionada la carga, el proceso arranca al validar las condiciones de seguridad. Las transiciones entre etapas se gestionan mediante contadores de tiempo alimentados por la señal de 1 segundo del Timer1 y contadores de ciclos implementados por software. Finalizado el secado, el sistema conmuta al estado *FIN* hasta un nuevo reinicio.

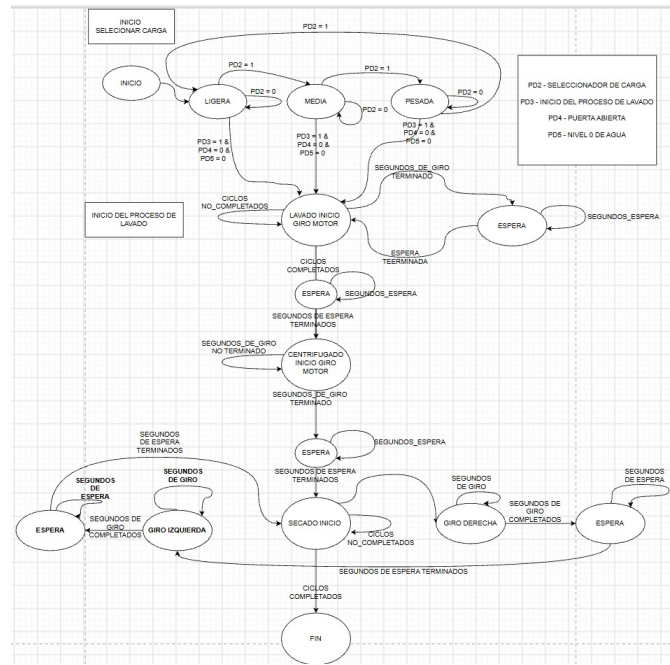


Figura 1. Representación del funcionamiento de la lavadora mediante la máquina de estado.

V-B. Problema B. Contador digital con tres botones y display de 7 segmentos

Para el desarrollo del contador digital se optó por una arquitectura basada en dieciséis bloques de código, uno por cada valor representable en el display (0 a 15, en formato hexadecimal), donde cada bloque es responsable tanto de mostrar el dígito correspondiente en el display de 7 segmentos como de evaluar las pulsaciones de los tres botones de entrada.

Las entradas del sistema son tres pulsadores conectados a los pines **PB0**, **PB1** y **PB2**, configurados como entradas con resistencias de pull-up internas habilitadas:

- **PB0:** incrementa el contador en uno, avanzando al bloque correspondiente al dígito siguiente.
- **PB1:** reinicia el contador a cero, retornando siempre al bloque `loop_cero` sin importar el valor actual.
- **PB2:** decrementa el contador en uno, retrocediendo al bloque correspondiente al dígito anterior.

Las salidas del sistema controlan un display de 7 segmentos, cuyas señales se distribuyen entre los pines **PD0-PD3** (cuatro segmentos) y **PC0-PC2** (tres segmentos), completando los siete segmentos necesarios. Cada uno de los dieciséis bloques enciende y apaga la combinación de segmentos correspondiente al dígito que representa, incluyendo los seis valores hexadecimales adicionales (10 a 15) más allá del rango decimal habitual.

Para evitar que el rebote mecánico de los pulsadores provoque lecturas erróneas, se implementó un mecanismo de confirmación basado en una bandera de software (bit 0 del registro `r18`). Al ingresar a cada bloque, si dicha bandera se encuentra activa —lo que indica que se acaba de producir un cambio de dígito por una pulsación— se invoca la rutina `confirmacion`, la cual introduce un retardo de aproximadamente 23 ms y luego llama a la rutina `debounce`,

que permanece en espera hasta confirmar que los tres botones fueron efectivamente liberados. Una vez estabilizada la entrada, la bandera se limpia y el flujo regresa al bloque correspondiente para continuar el conteo con normalidad. Este esquema evita que una única pulsación sea interpretada como múltiples cambios de estado consecutivos.

Dentro de cada bloque, tras mostrar el dígito en el display, se evalúa el estado de los tres pulsadores mediante instrucciones `sbis`. Si alguno de ellos se encuentra presionado, se activa la bandera de confirmación y se ejecuta el salto correspondiente (incremento, decremento o reinicio); en caso contrario, el flujo llega al final del bloque y regresa a su propio inicio, repitiendo la evaluación de forma continua hasta que el usuario realice una nueva acción.

V-C. Problema C. Selección y ejecución de secuencias

Se desea implementar un sistema de selección y ejecución de ocho (8) secuencias luminosas utilizando 8 LEDs, empleando un Arduino Uno programado en lenguaje ensamblador (ASM).

Para el desarrollo del sistema de selección y ejecución de secuencias se implementaron ocho patrones luminosos independientes (`Sec_1` a `Sec_8`), cada uno compuesto por una serie de valores que se envían secuencialmente al puerto de salida, generando distintos efectos visuales sobre los 8 LEDs (barrido simple, bloques deslizantes, convergencia desde los extremos, parpadeo tipo tablero de ajedrez, entre otros). La secuencia actualmente en ejecución se identifica mediante un registro dedicado, lo que permite navegar entre las ocho secuencias de forma circular: al superar la última secuencia se retorna a la primera, y viceversa. 2 Las entradas del sistema son tres pulsadores conectados a los pines **PB0**, **PB1** y **PB2**, configurados como entradas con resistencias de pull-up internas habilitadas:

- **PB0**: retrocede a la secuencia anterior.
- **PB1**: regresa directamente a la primera secuencia.
- **PB2**: avanza a la secuencia siguiente.

Las salidas del sistema son los 8 LEDs conectados al puerto A (**PA0-PA7**), configurado en su totalidad como salida. Cada secuencia controla estos LEDs de forma independiente, definiendo su propio patrón de encendido y apagado sin depender de un display u otro elemento de indicación adicional.

El aspecto más particular de la implementación es que la rutina de retardo utilizada entre cada paso de una secuencia cumple una doble función: además de temporizar la visualización, es también el punto donde se evalúa continuamente el estado de los tres pulsadores. Esto se logra verificando los botones en cada iteración del ciclo de espera más interno, lo que permite que una pulsación sea detectada de forma prácticamente inmediata, sin necesidad de esperar a que finalice el paso de la secuencia en curso. Al detectarse una pulsación, se actualiza el registro de secuencia activa según el botón presionado y se descarta de forma explícita la dirección de retorno pendiente en la pila — correspondiente a la llamada a la rutina de retardo interrumpida—, dado que el flujo de ejecución no retorna a la secuencia en curso sino que reinicia la evaluación desde la nueva secuencia seleccionada. Finalmente,

el sistema espera a que los tres pulsadores se encuentren liberados de forma simultánea, aplicando además retardos de filtrado (antirrebote) antes y después de dicha espera, para evitar que una única pulsación sea interpretada como múltiples cambios de secuencia consecutivos.

Cabe destacar que, dado que las secuencias se invocan mediante saltos incondicionales (`rjmp`) en lugar de llamadas a subrutina, la única dirección de retorno que llega a apilarse en algún momento es la correspondiente a la rutina de retardo, invocada esta sí mediante `rcall`. Al descartar dicha dirección tras detectar una pulsación, se evita que la pila acumule direcciones de retorno no utilizadas en cada cambio de secuencia. Esta limpieza se realiza extrayendo tres bytes de la pila, en lugar de dos, debido a que el microcontrolador utilizado (ATmega2560) dispone de más de 128 KB de memoria de programa, lo que obliga a representar cada dirección de retorno con 3 bytes en lugar de los 2 bytes habituales en microcontroladores AVR de menor capacidad. Esta es una consideración específica de la arquitectura del dispositivo empleado, y debería ajustarse a 2 bytes en caso de portar la solución a un microcontrolador con memoria de programa igual o inferior a 128 KB.

V-D. Problema D. Unidad Lógica Aritmética (ALU)

Se desea diseñar e implementar una Unidad Aritmético-Lógica (ALU) en Ensamblador para el microcontrolador Atmega2560. La ALU deberá operar sobre dos operandos de 4 bits y ejecutar diferentes operaciones lógicas y aritméticas de acuerdo con una señal de selección de 3 bits, siguiendo la tabla de funcionamiento presentada.

S2 S1 S0	Operación	Resultado (F)
000	CLEAR	00
001	A - B	A - B
010	A + B	A + B
011	XOR	A XOR B
100	AND	A AND B
101	OR	A OR B
110	SHL	A << 1
111	INC	A+1

Figura 2. Operaciones de la ALU

Para el desarrollo de la ALU se comenzó por definir la asignación de puertos de entrada. Dado que la cantidad de dipswitches disponibles no alcanzaba para representar simultáneamente dos operandos de 4 bits y una señal de selección de 3 bits, se optó por implementar un multiplexado

por software: los primeros cuatro bits de un mismo puerto representan el valor numérico a cargar, mientras que el quinto bit funciona como selector, determinando si dicho valor corresponde al operando A o al operando B.

Las entradas del sistema se distribuyen entre los puertos A y B, configurados como entradas con resistencias de pull-up internas habilitadas:

- **PA0-PA3:** valor numérico de 4 bits a cargar en A o B.
- **PA4:** bit selector, determina si el valor de PA0-PA3 se asigna al operando A o al operando B.
- **PB0-PB2:** código de operación (*OpCode*) de 3 bits, que selecciona la operación a ejecutar según la tabla de funcionamiento.

Como indicación adicional a nivel de hardware, se incorporó un LED en paralelo a cada dipswitch del *OpCode*, de modo que el usuario pueda identificar visualmente la operación seleccionada sin depender exclusivamente de la posición física de los switches.

Las salidas del sistema se implementaron mediante LEDs en los primeros cinco pines del puerto L (**PL0-PL4**), dado que el mayor valor representable por las operaciones definidas —considerando operandos de 4 bits— es 30, requiriendo 5 bits para su correcta visualización. Adicionalmente, se utilizaron los primeros tres pines del puerto G (**PG0-PG2**) para mostrar las banderas de estado Zero, Carry y Negativo.

El programa opera mediante un bucle principal (*main_loop*) que lee continuamente el valor del *OpCode*. Dado que las entradas se toman con lógica invertida (nivel bajo al presionar, por efecto de las resistencias de pull-up), el valor leído se complementa a uno y se enmascara para restringirlo a los tres bits efectivamente utilizados, dejando disponibles los bits restantes para una eventual ampliación futura del código. El valor resultante se compara secuencialmente contra cada una de las ocho operaciones posibles, saltando a la subrutina correspondiente en caso de coincidencia.

Dentro de cada subrutina de operación, los valores de A y B se obtienen mediante una subrutina auxiliar de selección, la cual evalúa el quinto bit del multiplexor para determinar a cuál de los dos operandos corresponde el valor leído en ese momento, actualizando el registro correspondiente. Antes de operar, los valores de A y B se copian a registros provisionarios, evitando así que las instrucciones aritméticas o lógicas modifiquen los valores originales de los operandos entre una iteración y la siguiente. Tras cada operación, se capturan las banderas Zero, Carry y Negativo directamente del registro de estado del microcontrolador (*SREG*) y se reflejan en las salidas correspondientes del puerto G. Adicionalmente, en cada iteración de las subrutinas se vuelve a comprobar el valor del *OpCode*, de modo que, si el usuario cambia la operación seleccionada, el sistema abandona la subrutina en curso y retorna al bucle principal sin demoras adicionales.

Dos operaciones presentan un comportamiento particular respecto al resto. En el caso del *OpCode* 000 (*clear*), los valores de A y B no se utilizan, dado que la operación consiste simplemente en limpiar la salida sin depender de los operandos. En el caso del *OpCode* 001 (*resta*), se incorporó una verificación adicional sobre la bandera de Negativo: si

esta se encuentra activa, se aplica complemento a 2 sobre el resultado antes de mostrarlo, de modo que el valor visualizado corresponda siempre al valor absoluto de la resta. De esta forma, el usuario puede interpretar correctamente un resultado negativo observando que la bandera de Negativo está encendida, sin necesidad de que la salida numérica se muestre directamente en complemento a 2.

VI. RESULTADOS

VI-A. Problema A. Lavadora Automática

Se logró un funcionamiento correcto de los tres modos de operación (lavado, centrifugado y secado), respetando los ajustes de tiempo correspondientes a cada tipo de carga seleccionada. Para la temporización de los distintos estados se optó por utilizar el periférico *Timer1* del microcontrolador en modo CTC, a pesar de que dicho tema aún no había sido abordado formalmente en el curso al momento de realizar este laboratorio, lo cual implicó un proceso de investigación adicional por fuera de los contenidos vistos en clase hasta ese momento.

Dado que no se contaba con los sensores físicos de puerta y de nivel de agua, ambas entradas se simulaban de forma manual mediante resistencias que se conectaban y desconectaban físicamente para representar los distintos estados de "puerta cerrada"/"puerta abierta", "nivel de agua correcto"/"nivel insuficiente".

Durante la revisión con el docente surgió además una observación relevante sobre una decisión de diseño inicial: originalmente se había planteado que el mismo pin encargado de activar el LED indicador del modo *lavado* encendiera también, en paralelo, el motor correspondiente. El docente señaló que, si bien esta disposición no estaba explícitamente prohibida por la consigna, sobrecargaría la corriente máxima que puede entregar de forma segura un pin de un microcontrolador estándar, dado que un motor real demanda una corriente considerablemente mayor a la que soporta una salida digital de propósito general. Esta observación llevó a reconsiderar la separación conceptual entre la señal de indicación (LED) y la señal de control del motor, un aspecto a tener en cuenta ante una futura implementación con hardware de potencia real (por ejemplo, mediante un transistor o relé de accionamiento).

VI-B. Problema B. Contador digital

La principal dificultad no estuvo relacionada con la lógica del programa en sí, sino con la adaptación al microcontrolador utilizado: al trabajar sobre un ATmega2560 en lugar del ATmega328P (Arduino Uno) originalmente indicado en la consigna, la mayor cantidad de pines y periféricos disponibles generó cierta confusión inicial al momento de seleccionar y configurar los puertos a utilizar. Superada esta etapa de adaptación, el contador funcionó correctamente, permitiendo incrementar, decrementar y reiniciar el valor mostrado mediante los tres pulsadores correspondientes.

VI-C. Problema C. Selección y ejecución de secuencias

De forma similar al problema anterior, la principal dificultad estuvo asociada a la adaptación de la solución al

ATmega2560. Una vez resuelta esta etapa inicial, las ocho secuencias luminosas funcionaron correctamente, permitiendo avanzar, retroceder y reiniciar la secuencia activa mediante los pulsadores correspondientes, sin pérdida de sincronismo entre pasos.

VI-D. Problema D. ALU de 4 bits

El sistema funcionó correctamente durante la etapa de simulación, lo cual generó inicialmente la impresión de que la implementación estaba finalizada. Sin embargo, al trasladar la solución al hardware físico se observó un comportamiento incorrecto: las combinaciones de entrada no se correspondían con la operación esperada según la tabla de funcionamiento (por ejemplo, para seleccionar la operación *clear* era necesario ingresar la combinación 111 en lugar de 000). Tras un análisis detallado se determinó que esta discrepancia se debía a una configuración incorrecta de la lógica de entrada en el entorno de simulación utilizado, que no representaba fielmente el comportamiento de las resistencias de pull-up en el hardware real. La solución adoptada consistió en incorporar instrucciones de complemento a uno (`com`) sobre los valores leídos de los puertos de entrada, invirtiendo la lógica leída y restableciendo la correspondencia esperada entre las combinaciones de switches y las operaciones de la tabla. Una vez aplicada esta corrección, el sistema funcionó correctamente tanto en simulación como en la implementación física.

En términos generales, y una vez resueltas las dificultades mencionadas, los cuatro sistemas desarrollados funcionaron correctamente de acuerdo con lo especificado en cada consigna. El código fuente completo, las simulaciones y los videos demostrativos de cada uno de los problemas se encuentran disponibles en el repositorio y el material audiovisual referenciados en la Sección VIII.

VII. CONCLUSIONES

El desarrollo de los cuatro problemas propuestos permitió cumplir con el objetivo general planteado al inicio de este informe, logrando un manejo práctico de los registros de propósito general, el registro de estado SREG, los puertos de entrada/salida, las operaciones aritméticas y lógicas, las instrucciones de salto condicional, y la implementación de retardos tanto por software como mediante temporizadores por hardware. Más allá de los contenidos técnicos, el laboratorio permitió experimentar de primera mano las diferencias entre el comportamiento simulado y el comportamiento real de un sistema, así como la relevancia de las decisiones de diseño a nivel de hardware en la implementación de soluciones basadas en microcontroladores.

VIII. MATERIAL COMPLEMENTARIO

El código fuente completo de los cuatro problemas desarrollados, junto con su historial de versiones, se encuentra disponible en el siguiente repositorio:

- **Repositorio GitHub:** <https://github.com/tiagoreino/Laboratorio-1.git>

Los videos demostrativos del funcionamiento de cada sistema en hardware físico, se encuentran disponibles en:

- **Videos (Google Drive):** <https://drive.google.com/drive/folders/1S9oluBCLg4OkxOA5pSfvMtGWvjA01kLX?usp=sharing>

REFERENCIAS

- [1] Microchip Technology, *ATmega328P – 8-bit AVR Microcontroller*, [En línea; consultado el 31-agosto-2026], 2024. dirección: <https://www.microchip.com/en-us/product/atmega328p>
- [2] Seeed Studio, *ATmega2560 – Features, Comparisons, and Arduino Mega Review*, [En línea; consultado el 30-agosto-2026], 2019. dirección: <https://www.seeedstudio.com/blog/2019/11/13/atmega2560-features-comparisons-and-arduino-mega-review/>
- [3] Microchip Technology, *Microchip Studio for AVR® and SAM Devices*, [En línea; consultado el 31-agosto-2026], 2024. dirección: <https://www.microchip.com/en-us/development-tool/microchip-studio>
- [4] Gamboa, L. C., *PICSimLab Documentation – Introduction*, [En línea; consultado el 31-agosto-2026], 2024. dirección: https://lcgamboa.github.io/picsimlab_docs/stable/Introduction.html
- [5] Microchip Technology, *Clear Timer on Compare Match (CTC) Mode*, [En línea; consultado el 31-agosto-2026], 2023. dirección: <https://onlinedocs.microchip.com/oxy/GUID-0EC909F9-8FB7-46B2-BF4B-05290662B5C3-en-US-12.1.1/GUID-A2D59471-CDAB-4F0A-87E5-EA98C4CAB777.html>
- [6] Robucci, R., *Lecture 04 – AVR CPU Registers*, [En línea; consultado el 31-agosto-2026], 2018. dirección: https://eclipse.umbc.edu/robucci/cmpe311/Lectures/L04-AVR_CPU_Registers/
- [7] All About Circuits, *Contact "Bounce" – Switches*, [En línea; consultado el 31-agosto-2026], 2021. dirección: <https://www.allaboutcircuits.com/textbook/digital/chpt-4/contact-bounce/>